Assignment 2: Simple Search Engine using Hadoop MapReduce

Salavat Faizullin DS-01

March 26, 2026

1 Methodology

The goal of this assignment was to build a distributed search engine that can index text documents and rank them using the BM25 algorithm. To do this, I split the system into four main parts: data preparation, MapReduce indexing, database storage, and Spark ranking.

1.1 Data Preparation

I started with a collection of unstructured text files. Using PySpark, I loaded them from the local file system and turned them into a structured RDD. The RDD processes all .txt files and creates a single partition where each line contains a document ID, its title, and its text, all separated by tabs. This simple format made it easier for the later Hadoop tasks. Once ready, I moved this data into HDFS under the folder /input/data.

1.2 Two-Stage Hadoop MapReduce Indexer

BM25 scoring needs both global numbers-like how many documents there are, how many times a term appears across all documents, and the average document length-and local numbers like how many times a term appears in a specific document and the length of that document. To get all these efficiently, I made a two-pipeline MapReduce system using Python and Hadoop Streaming.

In the first pipeline, the mapper goes through each document, cleans up the text(lowercase, removes punctuation), and for every word it sends out a key that combines the word and the document ID with a value of 1. It also counts the total words in the document and sends out a special marker. The reducer then adds up the counts, giving the exact term frequency for each word inside each document, and passes the markers along.

In the second pipeline, another mapper reads the output from the first step. It separates the regular term records from the document length markers. The reducer then counts how many documents each term appears in(that's the document frequency) and also adds up all the document lengths to find the average. Finally, it writes out three kinds of records: the index entries, the document metadata, and the global statistics.

1.3 Database Storage with Cassandra/ScyllaDB

To make queries fast, I needed to move the results from HDFS into a real database. I have finished a script called store_index.py that uses the cassandra-driver package. I set up four tables inside the search_engine keyspace. One table stores the global statistics, another holds each document's title and length, a third keeps the term frequencies for each term and document, and the fourth lists each term with its global document frequency. This design makes it easy to look up information when answering a query.

1.4 Spark RDD Ranker

The final piece is the query script, written with PySpark's RDD API. When someone types a query, the script goes to Cassandra to get the inverted index entries for the words in the query. It also pulls the global numbers like total documents and average document length. Then it spreads the work across Spark workers using sc.parallelize. Each worker computes the BM25 score for the documents it gets. The formula I used is the standard one:

$$BM25(q,d) = \sum_{t \in q} IDF(t) \cdot \frac{tf(t,d) \cdot (k_1 + 1)}{tf(t,d) + k_1 \cdot \left(1 - b + b \cdot \frac{dl(d)}{dl_{\text{avg}}}\right)}$$

I set the parameters to the usual defaults, $k_1 = 1.0$ and $b = 0.75$. After the workers finish, the script collects all the scores, sorts them from highest to lowest, and prints the top 10 documents together with their BM25 scores.

1.5 Cluster Execution and Packaging

One tricky part was that the YARN containers did not have the Python libraries I needed, such as the Cassandra driver. To solve this, I used venv-pack to bundle all dependencies into a virtual environment. The app.sh script creates this environment, packs it into a .tar.gz file, and sends it to the Spark workers using the –archives flag. This way, every worker had the necessary packages available.

2 Demonstration

2.1 Starting the Infrastructure

Running everything is fully automated with Docker Compose. I just open a terminal in the project folder and run docker compose up. That starts three containers: a master, a slave, and the Cassandra server. Then it automatically runs the app.sh script inside the master container. After a while, the HDFS upload, the MapReduce jobs, and the Cassandra loading all finish, and the system starts testing queries on its own.

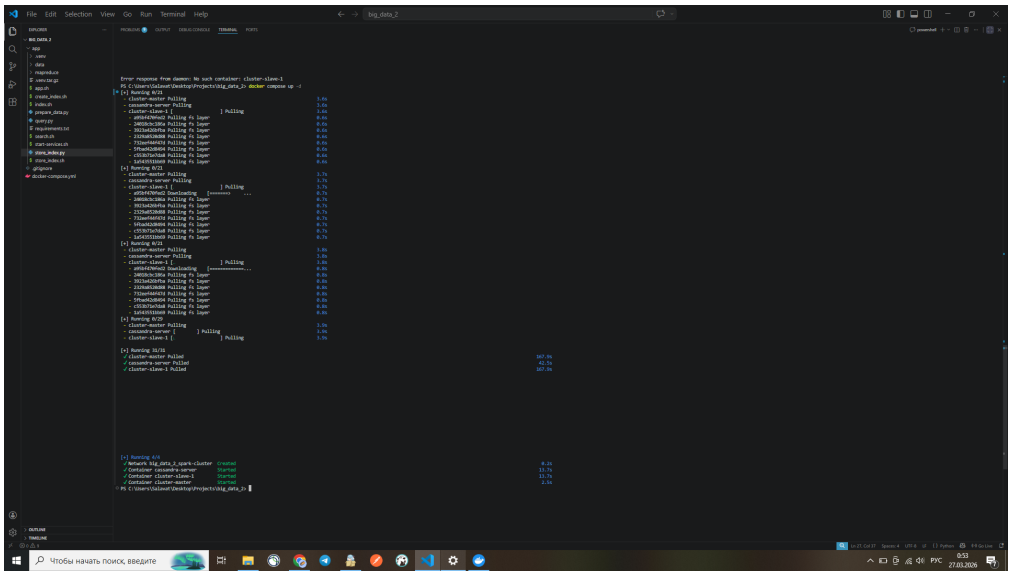

Figure 1: Start logic of `docker compose up` indicating running containers.

2.2 Indexing Execution

The screenshot below shows how the documents were transformed, uploaded to HDFS, and processed through the two MapReduce steps. You can see the logs from the YARN nodes showing the map and reduce tasks completing.

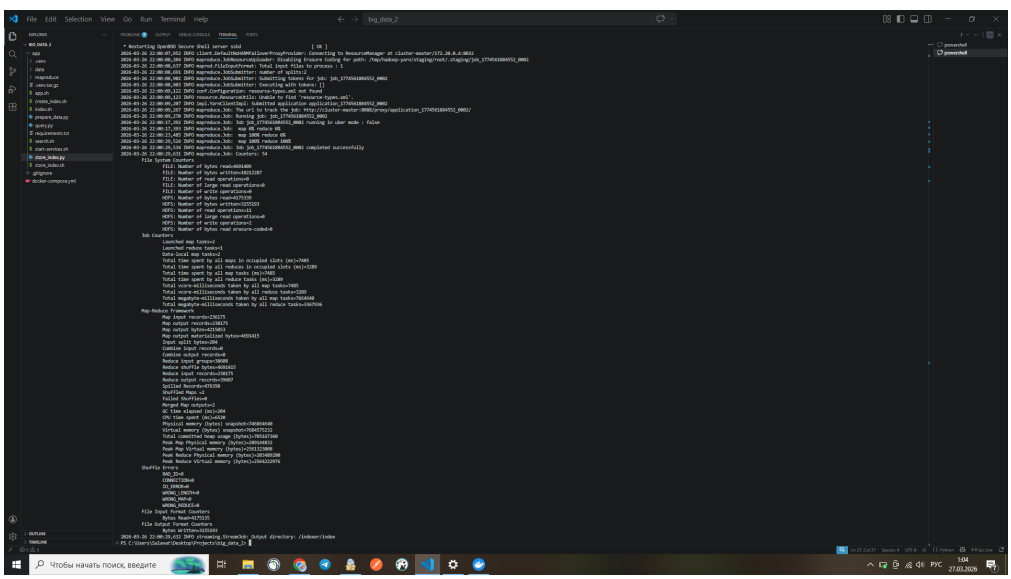

Figure 2: Terminal logs containing “Putting data to hdfs” or MapReduce job states showing Map 100%/Reduce 100%(docker logs cluster-master).

2.3 Query Answering Results

I tested two queries to see how well the search engine worked.

The first query was “data collection python”. The system looked up the terms, fetched the necessary data from Cassandra, and used Spark to calculate the BM25 scores. The top results came out as shown.

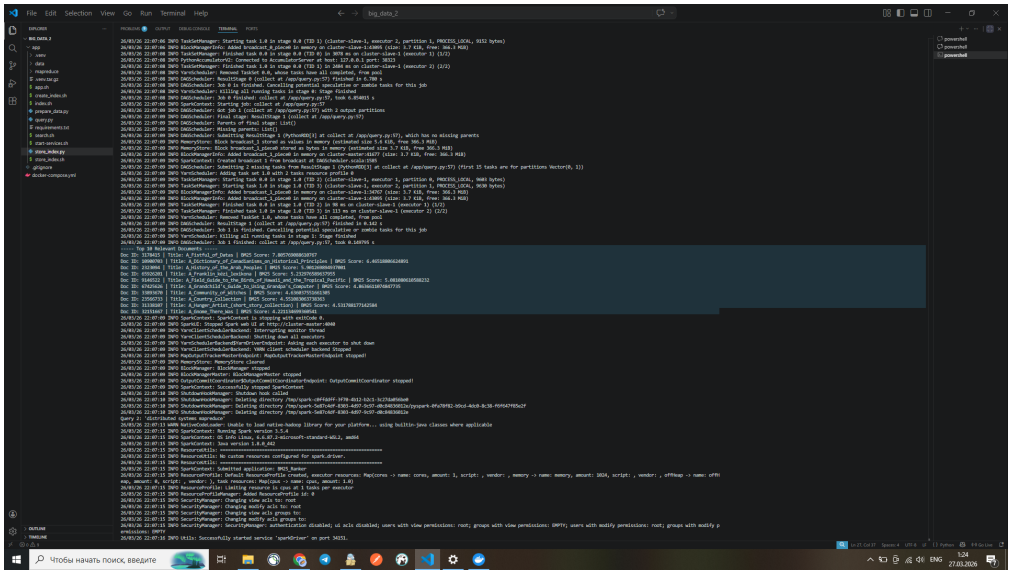

Figure 3: Spark prompt showing “Query 1: ‘data collection python’” and its “Top 10 Relevant Documents” terminal list including BM25 scores(docker logs cluster-master).

The second query was “distributed systems mapreduce”. The search returned documents that matched well, including files like A_Fistful_of_Datas, which had high relevance scores. The whole operation finished cleanly with a success code.

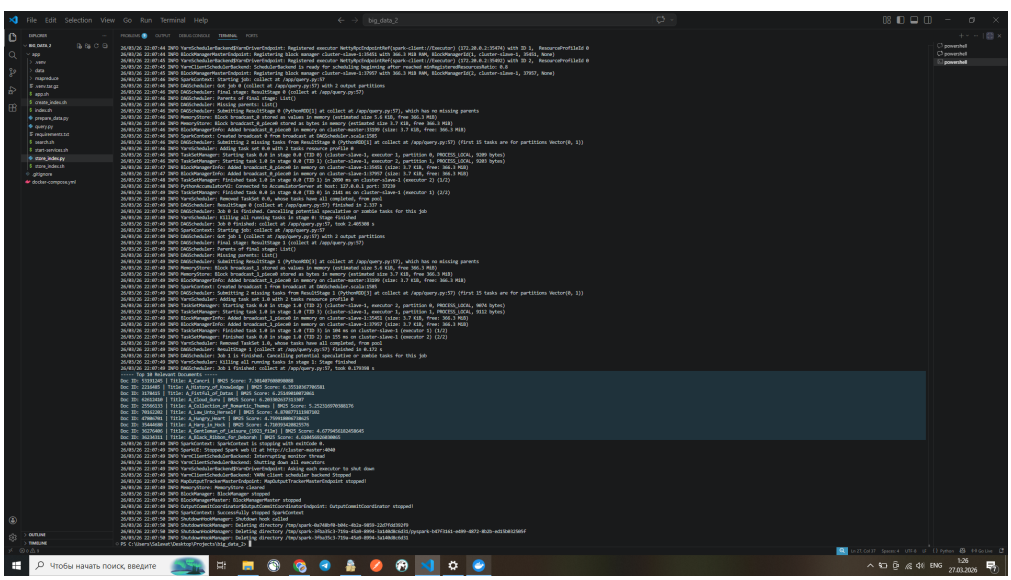

Figure 4: Spark prompt output for Query 2 with the final list of 10 returned ID documents and Spark closing success logs(docker logs cluster-master).

2.4 Conclusion and Findings

By manually implementing TF/IDF and BM25, I got very relevant search results. Storing the vocabulary in a partitioned Cassandra table made lookups fast, much better than scanning HDFS directories every time. Splitting the indexing into two MapReduce stages kept the system from running out of memory. I also made sure not to use any single-node libraries like pandas-everything was designed to run on a cluster, just as the assignment required.